

Keeping Delivery Promises at Olist

IT5006 Phase 3 Final Report

AY2026/27 Semester 1

Group 6

CHAI WEI NI, WINNIE (A0172261M)
LE TRUNG HIEU (A0286973H)
LIANG ZIYAN (A0291506E)
LUM ZI YUAN JOSHUA (A0108499N)
SHAH PRATIK (A0211041H)
YUAN JIAYI (A0177893H)

Code and notebooks: https://github.com/YuanJiayi/Team6_IT5006_Ecommerce_Analytics_AY2627Sem1

Deployed app: https://olist-ops-e5abckh8h0hfahdp.eastasia-01.azurewebsites.net/

Executive Summary

Olist's delivery control center needs two things from its data: a reliable delivery promise at checkout, and an early warning for parcels likely to miss that promise once they reach the carrier. We study 96,272 eligible delivered orders from the public Olist dataset, choose every model on time-ordered validation windows, and compare each one with the simple rule an operations team would otherwise use.

The strongest result is the promise. Setting each promise from our checkout estimate plus a regional buffer recalibrated every month gives 21.6-day promises with 1.9% of orders late on 19,363 later orders, against Olist's 22.1 days and 3.5%: shorter promises, and fewer of them missed.

For the late warning, a 50/50 blend of a regularised logistic model and the remaining-slack rule ranks late parcels as well as the rule and finds them earlier. In a daily replay it reaches 256 late parcels with 6.7 days of warning on average, against 300 parcels at 4.4 days for the rule. That leaves time to escalate with the carrier as well as warn the customer.

Deliveries became much faster during the test period. The ridge delivery-time estimates, chosen on validation, did not beat their route rules on the test months: at checkout the two are level (MAE 5.114 against 5.199 days), and at handover the rule is better (3.762 against 4.199 days). Monitoring is built to catch this kind of drift.

The models run behind a dashboard with a daily action list, an order-level prediction service and monthly model checks. We recommend piloting the promise policy in selected regions with a peak-season guard, keeping the monthly refit and model checks in place, and widening the rollout once the pilot confirms the gains.

1 Literature Review

Our Phase 1 review shaped three decisions: what to predict, which information to predict it from, and how to judge the result in operational terms.

Delivery promises matter to customers. Deshpande and Pendem analyse more than 15 million orders from Tmall and the Cainiao logistics network and find that customers rate delivery against the speed they were promised [5]. Salari et al. use JD.com data to turn tree-based delivery-time distributions into a cost-sensitive promise policy [4]. These studies lead us to predict the delivery time itself and whether a parcel will miss its promised date.

Model choice. Grinsztajn et al.'s benchmark of 45 datasets finds that tree-based methods remain strong on medium-sized tabular data [7], so forests and boosting are serious candidates here. We compare them with plain linear and logistic regression and single decision trees, and keep a more complex model only if it improves validation performance enough to justify the extra cost of running and explaining it [2].

Features and prediction timing. De Araujo and Etemad predict last-mile delivery times for 3,253,252 Canada Post deliveries in the Greater Toronto Area from origin–destination coordinates, Haversine distance, calendar inputs and weather [8], which supports our distance and calendar features. Salari et al.'s finding that fulfilment operations matter for delivery forecasting supports our order-content and seller inputs [4]. Küp et al. split delivery lead time into processing, handling and shipping durations between operational checkpoints [9]. At checkout those durations are still unknown, so we use them only in the handover model, once the earlier stages are complete. Historical summaries use only earlier outcomes, and preprocessing

is fitted within training folds.

Evaluation. Bergmeir and Benítez show that time-ordered data need evaluation designs that respect the order of observations [1], so we validate on expanding chronological windows. Hodson shows that MAE and RMSE suit different error distributions and that neither is better in general [10]. We select on MAE, which reads directly in days, and report RMSE to show the larger misses. For the minority class of late deliveries, Saito and Rehmsmeier show that precision–recall analysis reveals weaknesses that ROC plots hide [3], so we select on average precision (PR-AUC) and report precision and recall at a fixed review capacity. Following Gaudreault and Branco's comparison of metrics for imbalanced classification, we also report MCC [11].

2 Dataset and Preprocessing

2.1 Data source

We use the Olist Brazilian E-Commerce Public Dataset. It records 99,441 orders placed between September 2016 and October 2018 on Olist, a marketplace connecting small Brazilian sellers to large retail sites. The data are split across nine tables (Appendix Table 20). Orders link to customers through `customer_id`, to items, payments and reviews through `order_id`, and through items to products and sellers. ZIP-code prefixes link customers and sellers to the geolocation table. Each order carries purchase, approval, carrier-handover and customer-delivery timestamps and Olist's estimated delivery date, which is the promise shown to the customer and the reference for lateness.

Most orders have a delivered status (96,478), and 96,470 of them also have both delivery dates and enter the delivered cohort (Table 1). The rest were still in transit, cancelled, unavailable or being processed at extraction. Review scores are outcomes recorded after delivery, so they motivate the problem (Section 3) but never enter a model.

2.2 Cleaning and integration

Orders are joined to customers, items, products, sellers and payments, with item and payment rows aggregated per order. Where an order involves several sellers, we use the one furthest from the customer. Geolocation points are reduced to the median coordinates of each ZIP prefix after removing 42 implausible points, so the many observations per prefix do not multiply order rows. The 610 products without a category are kept as an "unknown" category (1,374 modelling orders), and the 16 orders containing a product without weight or dimensions have those values imputed within each training fold. In 1,359 orders across the dataset the carrier handover is recorded before payment approval, a logging inconsistency in the source data. Of these, 1,185 are in the modelling cohort, where we treat approval time as unknown at handover and keep the order. Another 165 record the handover even before the purchase and fall among the excluded orders below, and the remaining 9 were never delivered. For the 547 orders with several review records we use the lowest score. Among the 95,627 reviewed modelling orders, using the latest review or mean score changes the low-rating share by less than 0.1 percentage points.

Of 99,441 recorded orders, 96,470 were delivered with both actual and estimated delivery dates (Table 1). We exclude 198 without a valid purchase–handover–delivery sequence, 165 of them with a handover recorded before the purchase, leaving 96,272 orders for the principal model comparisons. Of these, 75,039 had outcomes recorded before the 26 May 2018 cutoff and 19,230

were purchased on or after it. The 2,003 earlier purchases whose outcomes were not yet recorded at the cutoff are excluded from the initial fitting pool and test cohort. Section 5.2.1 defines the broader promise-policy cohort. An order is late if it arrives after Olist's estimated delivery date; arrival on that date is on time.

Table 1: Order cohorts.

Cohort	Filter	Orders	Customers
All orders	None	99,441	96,096
Phase 1 EDA	At least one item, any status	98,666	95,420
Delivered	Actual and estimated delivery dates present	96,470	–
Modelling	Valid purchase–handover–delivery sequence	96,272	93,159
Fitting pool	Outcome recorded before 26 May 2018	75,039	–
Test	Purchased on or after 26 May 2018	19,230	–
Excluded	Outcome not recorded by 26 May 2018	2,003	–

Customer counts. Customers are counted by `customer_unique_id`, which identifies a person across orders. `customer_id` is issued afresh for every order and would make each order look like a new customer. The Phase 1 figure of 95,420 unique customers comes from the 98,666 orders with at least one item, of any status, and the 96,272 modelling orders belong to 93,159 unique customers.

2.3 Feature engineering

All models start from 20 inputs known when the customer pays, grouped as promise, route, seller history, order contents, payment and timing (Appendix Table 10). Models used at carrier handover add inputs that only become known then (Appendix Table 11). These are introduced with each model in Sections 5.2.2 and 5.3.

History features. Three inputs summarise earlier orders. The route's typical delivery time is the mean purchase-to-delivery time on the same seller-state–customer-state route, and the seller's typical shipping time is the mean purchase-to-handover time of that seller's earlier orders, ignoring values outside 0–60 days. Both use only orders whose outcome was recorded before the current purchase, and both are smoothed towards the overall mean so routes and sellers with few orders are not set by a handful of deliveries (prior strengths 20 and 10). Promise slack, Olist's promised days minus the route's typical delivery time, shows how generous the promise is for that route. Purchase month was tested and removed because the training data contain at most one earlier year for each month, too little to learn a seasonal effect.

Order size and value. Freight, weight and volume describe the parcel the carrier has to move. Freight is the shipping charge quoted at checkout and already prices in package size and distance. Weight and volume add the physical size, since heavy or bulky parcels are handled and routed differently. Price stands in for the kind of product and seller, such as furniture or electronics versus small accessories. In the checkout ridge model (Section 5.2.1), freight has the third-largest standardised coefficient among the numeric inputs, averaged over the five windows (0.87 days per standard deviation), behind only promised days and distance. Price, weight and volume contribute less (0.03–0.14), as freight already carries most of their shared signal.

2.4 Data validation and leakage control

Validation of the prepared table. A Great Expectations (GX Core 1.24.0) suite checks the 96,470-row prepared order table before the sequence exclusions, and all 35 expectations pass. For the fixed final models, timestamp checks confirm that fitting outcomes were recorded before 26 May 2018. Monthly-refit candidates use only outcomes recorded before each refit date.

Prediction points and point-in-time features. Each model is evaluated at the moment it would be used: the delivery-time estimate at checkout and at carrier handover, and the late warning at handover. A feature is allowed only if its value was known then. Route delivery histories use completed deliveries, seller handover histories use recorded handovers, and backlog histories count orders still open at each snapshot, excluding the target order. An order's own eventual delivery date, review and later status are never predictors. Imputation, encoding and, where applicable, scaling are fitted inside scikit-learn Pipelines on each fold's fitting rows only, so validation and test rows never inform preprocessing. Checkout availability of recorded promises, payment details and static geolocation is assumed because historical snapshots are unavailable.

3 Problem Definition and Stakeholder Context

3.1 Business problem

Olist shows every buyer an estimated delivery date at checkout, and customers judge the delivery against that promise [5]. Of the 96,272 orders in the modelling cohort (Table 1), 6.8% arrived after it. Lateness is strongly associated with low ratings. Among 95,627 delivered orders with a review, 62% of late orders received a one- or two-star review against 9% of on-time orders, which makes a late order 6.7 times as likely to earn one. Late orders account for 32% of all low ratings. Customers whose first order arrived late also bought again less often (2.5% against 3.0%).

Two features of delivery shape the problem. First, the promise is long. It averages 23.7 days against 12.6 days of actual delivery, and 79% of orders arrive at least a week early. A long promise protects the late rate but shows customers a slow date. Second, most uncertainty arises after the seller hands the parcel to the carrier. The carrier leg averages 9.3 of the 12.6 days and accounts for 85% of the variance in total delivery time. A date set at checkout therefore cannot remove all risk, but parcels that fall behind can still be found after handover, while there is time to act.

We address one problem, order delivery lead time, with two framings:

- **Regression:** predict total days from purchase to delivery, first at checkout and again at carrier handover (Section 5.2).
- **Classification:** predict whether a parcel will arrive after its promised date, scored at handover and used to rank parcels for review (Section 5.3).

Phase 1 also considered predicting low ratings directly. We set it aside because the strongest signal, how late the parcel arrives, is known only after delivery, and the signals available earlier covered few low-rated orders. Operations can act on lateness against the promise, not on the rating.

3.2 Stakeholders and decision points

The models feed a delivery control center at three decision points, each a moment when staff can still act (Table 2). Section 8 covers their use and expected impact.

Table 2: Decision points, model outputs and owners.

Decision point	Model output	Decision supported	Owner
Checkout	Delivery-time estimate without Olist's promise, plus a regional buffer	Show a revised promised date that is shorter or more reliable than Olist's	Commercial and customer experience
Carrier handover, reviewed each morning	Late-risk ranking (model blended with the remaining-slack rule) and expected days remaining	Rescue at-risk parcels: ask the carrier to expedite; warn the customer early when a parcel will clearly be late	Logistics operations and customer service; sellers when their handover caused the delay
Monthly	Forecast bias, late rate, promise length	Recalibrate the promise buffer; refit the classifier; switch models only when a refit proves better	Data team

Success criteria. Each output must beat what operations could do without machine learning. Models are selected on time-ordered validation windows and evaluated on the later test period (Section 5.1):

- **Delivery-time estimate:** lower mean absolute error (MAE, in days) than the route's average delivery time at checkout, and than days elapsed plus the route's recent carrier time at handover.
- **Late warning:** at the same review capacity, rank more late parcels near the top than the remaining-slack rule (Section 5.3), measured by PR-AUC (defined in Section 5.1) and the share of late orders caught in the top 10% of each list, and reach them with more days left to act. Accuracy is not used because it rewards predicting every order as on time.
- **Revised promise:** shorter promises than Olist's at the same late rate, or a lower late rate at the same promise length.

4 Exploratory Analysis Highlights

Phase 1 surveyed all nine tables. Some of its patterns on marketplace growth, customer reactions, delivery time and promise setting run through every later choice. Delivery times, late rates and review shares below are recomputed on delivered orders with the calendar-date late definition (Section 2.2). Correlations and other statistics are those reported in Phase 1, which compared full timestamps.

A growing, concentrated marketplace. Orders rose 137% from January–August 2017 to the same months of 2018, peaking at 7,451 orders with items on Black Friday in November 2017. Growth came from more orders, not larger baskets, since average order value (R$137.75) and items per order (1.14) barely changed. São Paulo accounted for 41.9% of orders, and the top 10% of sellers generated 67.6% of product sales. A marketplace changing this fast should

not be evaluated on a random split, so models are evaluated on later orders in time-ordered windows (Section 5.1).

Late delivery is associated with low ratings. Orders more than a week late received a one- or two-star review 79% of the time, against 9–11% for orders arriving on or before the promised date; arriving very early barely helps (Figure 1B). In Phase 1, lateness correlated far more strongly with the review score ($r = -0.365$) than route distance did ($r = -0.058$). The late warning therefore predicts arrival after the promised date, rather than a long delivery as such.

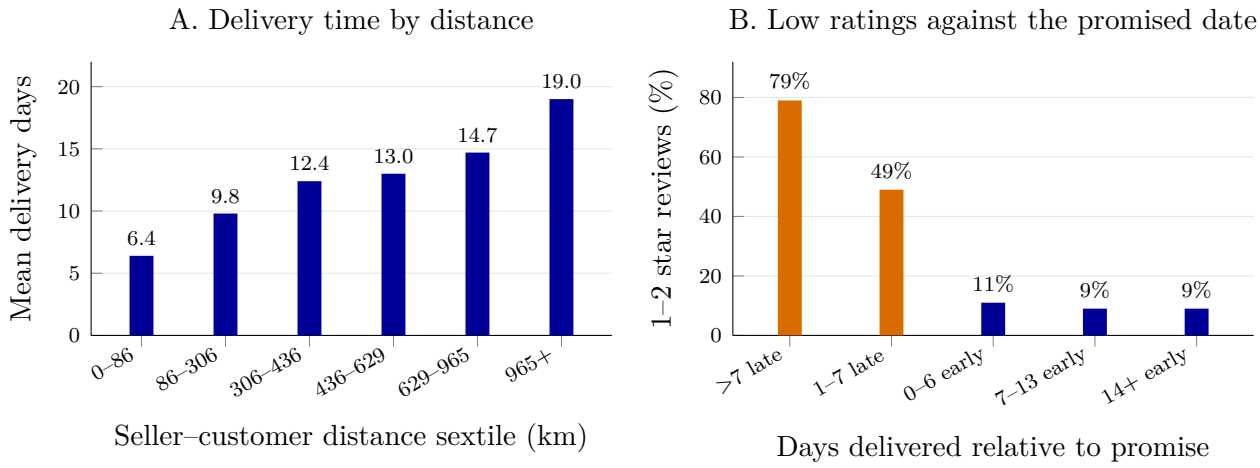


Figure 1: A: mean delivery days by distance sextile. B: share of one- or two-star reviews by delivery date relative to the promised date; orange bars are late orders, and "0–6 early" includes arrival on the promised date. Delivered orders, 2016–18.

Distance and route explain much of delivery time. Mean delivery time rises from 6.4 to 19.0 days across distance sextiles (Figure 1A), while the late rate rises more gently, from 4.4% to 10.2%, consistent with Olist lengthening the promise for distant routes. Distance, state and route history enter every model (Section 2.3).

The carrier leg dominates. Shipping after handover makes up about 74% of mean lead time (Section 3), and it was the only stage whose duration rose with order volume, suggesting a carrier-stage bottleneck under load. Most of the uncertainty is still ahead when the seller ships, so a second forecast at carrier handover, once elapsed time and recent route transit are known, targets the stage that matters most.

Timing, payment and order complexity matter. Friday orders took 14% longer than Sunday orders, in the seller handling stage. Boleto payments took about 1.2 days longer than card payments, in payment processing. Multi-seller orders were faster than single-seller ones (9.1 against 12.6 days), and weight showed only a weak relationship. Purchase timing, payment type and order contents are known at checkout and kept as inputs. Salari et al. [4] likewise show the value of including fulfilment operations in delivery-time forecasting.

Backlog lingers after demand peaks. After Black Friday, weekly orders peaked at 2,915, but the open-order backlog kept growing to 4,429 the following week, and median delivery time stayed elevated for six weeks. Weekly order volume itself correlated only weakly with delivery time ($r = 0.13$), consistent with persistent congestion. The late warning's regional backlog input, which compares open orders with their recent level, targets this build-up (Section 5.3).

The promise is padded and moves with past performance. Olist's promised window tracked a trailing eight-week average of actual delivery time ($r = 0.45$) more than live backlog ($r = -0.26$), consistent with promise setting that follows past delays more than current backlog. From March 2017 the monthly mean promise sat at 22–28 days while actual delivery averaged

11–17 days (Figure 2). In mid-2018 both moved: by August, actual delivery had fallen to 7.7 days and the promise to 15.2 days, in the period that became our test set. The padding is the room a revised promised date can work in. The shift is the drift that later separates validation from test results (Section 6.3), and it is the reason the promise buffer is recalibrated monthly.

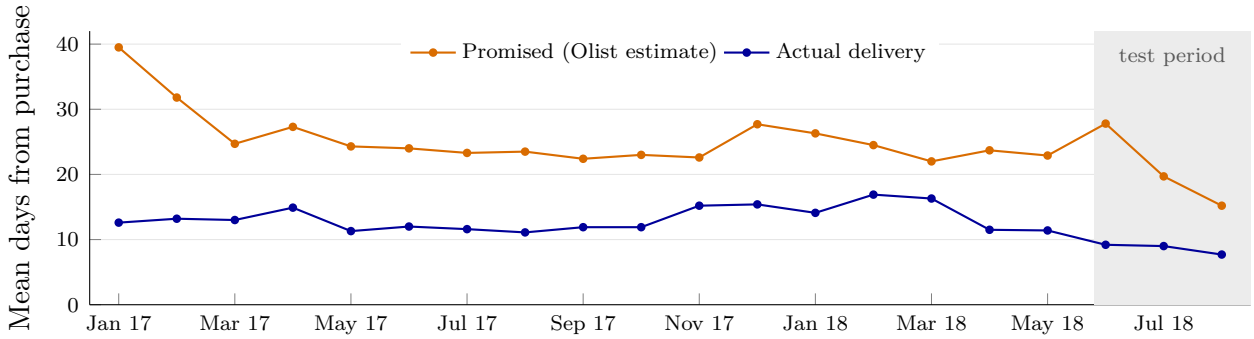


Figure 2: Monthly mean promised and actual delivery days by purchase month, January 2017 to August 2018. The shaded band marks the test period (purchases from 26 May 2018).

Useful negative results. A freight-to-price ratio showed a flat late rate across every bucket, so it was not built. For low-rating prediction, the strongest early signals covered few cases (multi-item orders were only 19.3% of low-rated orders), so the project models delivery lead time.

5 Modelling Approaches

5.1 Evaluation design

All models follow the point-in-time rules in Section 2.4.

Chronological split. Within the eligible delivered-order cohort with valid handovers, every order purchased on or after 26 May 2018 forms the test set (26 May–August 2018). The fixed-model comparison uses fits based on eligible outcomes recorded before that date. Monthly-refit candidates use eligible outcomes available strictly before each update. Earlier purchases supply five expanding validation windows (W1–W5, 22 September 2017 to 26 March 2018, about 7,600 orders each). Each window is fitted only on purchases whose delivery outcome was recorded before the window begins, so the fitting set grows from 23,664 to 52,166 orders (Table 3). Validation purchases stop 60 days before the test cutoff so slow deliveries near the boundary have time to arrive. We do not stratify, so each window keeps its natural late rate, which ranges from 4% to 21%. Chronological evaluation preserves the outcome availability this prediction task requires and tests changing delivery conditions. Time-series validation also needs explicit attention to temporal dependence [1].

Table 3: Expanding validation windows shared by the regression and classification models.

Window	Fitting orders	Validation orders	Late rate
W1	23,664	7,600	4.1%
W2	30,956	7,601	13.8%
W3	36,165	7,601	4.8%
W4	44,982	7,601	9.9%
W5	52,166	7,600	20.7%

Metrics. The regression is selected on mean absolute error (MAE) in days, the average size of a forecast error. RMSE, R^2 and mean signed error (prediction minus actual duration) are reported alongside. The classifier is selected on PR-AUC, implemented as average precision. It summarises precision and recall across review-list sizes and, unlike accuracy, does not reward predicting every order as on time when most orders arrive on time [3]. Selection metrics are averaged equally across the five windows. Because staff can review only part of the handovers, we also report recall and precision in the top 10% of each ranked list, within each validation window and, for the test set, within each purchase month. A daily replay, in validation and test, re-ranks all open orders each morning at the same review capacity (Section 5.3). The window's late rate is the no-skill precision–recall reference.

Model selection. We use a paired-window adaptation of the one-standard-error simplicity principle [2]. Candidates are ordered from simplest to most complex, and we choose the simplest one whose gap to the best is no larger than the standard error of that gap. The gap is paired by window, so its standard error is $s_d/\sqrt{5}$, where s_d is the standard deviation of the five per-window differences. For the regression, all configurations enter one ordered list (Section 5.4). For the classifier, each family is first tuned by its mean validation score. The rule is then applied to the tuned candidates and each family's plain baseline.

Test-period uncertainty. Model and operational-rule predictions are compared on the same test orders. We draw 2,000 paired bootstrap samples of the test period's 15 purchase weeks, with replacement and seed 42, retaining all orders in each sampled week. Each draw recomputes both methods' MAE for regression or average precision for classification and records their paired difference. The 2.5th and 97.5th percentiles of these differences define the reported 95% interval. Predictions are fixed, with no refitting inside the bootstrap. For ranked lists the monthly review list is rebuilt, while daily-queue comparisons resample review-morning weeks. An interval containing zero means no clear difference was detected.

Baselines. Each model is compared with a simple alternative available to operations. At checkout this is the route's average delivery time so far. At handover it is days so far plus the route's recent carrier time. For the late warning it is remaining slack, which ranks parcels by the days left until the promised date minus the route's recent carrier time. Closest to deadline and the regression's predicted lateness are further comparators (Section 5.3). The training mean is reported as a floor for the regression.

5.2 Regression: delivery-time estimate

Both stages predict the same target, total days from purchase to delivery, on the same orders, so their difference measures what the handover information is worth. At handover, remaining time equals the predicted total minus elapsed days. Numeric missing values use training-fold medians and categorical missing values form their own level. Linear models standardise numeric

inputs, apply $\log(1+x)$ to price, freight, distance, weight and volume, encode purchase hour as a sine–cosine pair and one-hot encode categories. Trees and forests use the same imputation and one-hot encoding without transformation or scaling, and gradient boosting uses ordinal category codes. Candidate models and their settings are given in Section 5.4.

5.2.1 Checkout

The checkout estimate is made when the customer pays, before the seller has shipped anything. It is the benchmark for the handover update, and it could be the starting point for a new promised date, which Section 8 evaluates as a recommendation.

Inputs. The model uses the 20 inputs defined in Appendix Table 10. Two describe Olist's promise (promised days, promise slack), five the route (distance, same-state flag, the route's typical delivery time, seller state, customer state), one the seller's typical shipping time, eight the order's contents (item, product and seller counts, total price, freight, weight, volume and product category), two the payment (type and instalments) and two the purchase time (hour and day of week).

Why order size and value should predict delivery time. Four of these inputs describe the parcel rather than the route, and each has a reason to affect delivery time.

- **Freight** is the shipping charge quoted at checkout. Carriers price a shipment from its size and the distance it travels, so freight summarises how far and how bulky the parcel is in one number.
- **Weight and volume** measure the parcel directly. Heavy or bulky items may need different packing, handling or carrier services. This is an a priori rationale, not proof of a causal effect in Olist.
- **Price** stands in for the kind of product and seller. Furniture or appliances differ from small accessories in packing, stock and dispatch, and high-value items may wait for additional payment or fraud checks.

The validation results support the role of freight more than the others. Removing all four inputs from the selected ridge raises validation MAE by 0.015 days (paired SE 0.010), worse in four of five windows. Freight accounts for most of this: removing it alone costs 0.011 days, while removing price, or weight and volume together, changes MAE by less than 0.003. The other size and value inputs add little signal once freight is included.

Promise-free variant and policy cohort. A forecast used to set a promise must not depend on the promise it replaces. The separate policy experiment therefore drops both promised days and promise slack, leaving 18 checkout inputs, and uses ridge with $\alpha = 1000$ and no separate search. Its broader delivered-order cohort does not require a valid handover: 38,005 validation and 19,363 test orders, compared with 38,003 and 19,230 in the principal model comparisons. Validation MAE changes from 5.758 to 5.778 days when the promise inputs are removed.

Calendar promise construction. For month M, the regional buffer is the 96.5th percentile of actual calendar-day duration minus ETA among purchases in $M-2$ delivered before M begins. Regions with fewer than 100 eligible orders use the pooled quantile. The new promised duration is $\lceil \text{ETA} + \text{buffer} \rceil$ calendar days after the purchase date. The headline replay updates the buffer monthly with the checkout model held fixed. Slow pre-cutoff purchases enter later calibration cohorts once their outcomes become available.

5.2.2 Carrier handover

At carrier handover the model keeps the 20 checkout inputs, adds observed elapsed and approval durations, and refreshes recent route transit (Appendix Table 11):

- **Elapsed days:** purchase-to-handover time already spent.
- **Approval days:** payment-processing time within that duration. Approval after handover is treated as unknown.
- **Recent route transit:** carrier time from deliveries completed on the same seller-state–customer-state route in the preceding 90 days. Sparse routes are smoothed towards the historical overall mean (prior strength 20).

5.3 Classification: late-order warning

Decision supported. When a seller hands a parcel to the carrier, operations can still ask the carrier for a faster service or warn the customer before the promised date passes. Staff can review only part of each day's parcels, so the late warning ranks them by their risk of arriving after Olist's estimated delivery date (Section 2.2). A ranking is useful if late parcels reach the top of the list with enough time left to act.

Class imbalance. Late rates range from 4% to 21% across validation windows and are 3.5% in the test set. We keep the natural late rate so that scores reflect real conditions, and handle the imbalance through the choice of metric. Class weighting was compared against the unweighted model on validation windows that keep their natural late rates. With $C = 0.01$, balanced class weights lower mean PR-AUC from 0.392 to 0.385, and they lower it at every C from 0.003 upwards. Only at the strongest penalties ($C \leq 0.001$) do they score slightly higher, still below the selected model (Appendix B), so the unweighted model is kept. A likely reason is that weighting mainly shifts predicted probabilities upwards, which matters for a threshold but little for a ranked list.

Inputs. The classifier uses the 23 handover inputs of Section 5.2.2 and four more that describe the time left and current delivery pressure (Appendix Table 11). Remaining slack is the days left until the promise minus the route's recent carrier time. Two inputs describe what happens once slack runs out: whether the handover timestamp exceeds the promise's midnight timestamp, and the slack shortfall, the number of days slack falls below zero. Lateness jumps at that point, and these inputs let a linear model represent the jump. Relative buyer-state backlog compares the state's open orders with their level at eight earlier weekly snapshots, as a signal of unusual pressure on delivery capacity.

Comparators. Two rankings need no model. The remaining-slack rule (Section 5.1) reviews the smallest slack first and is the formal benchmark. *Closest to deadline* reviews the fewest days left until the promised date first. It is what operations might do without any delivery history, because it ignores the route. A third ranking reuses the handover ridge of Section 5.2.2 ($\alpha = 1000$): parcels whose predicted delivery runs furthest past the promise come first. It tests whether a separate classifier is needed at all. The three comparators share one form, days left until the promise minus the delivery time still expected, and differ only in that expectation: none for closest to deadline, the route's recent carrier time for the rule, and a full model estimate for the regression ranking.

Why PR-AUC. PR-AUC (average precision) scores the late-order review ranking under imbalance [3]. It depends on prevalence, so rankings are compared only within the same period and on the same orders. ROC-AUC and fixed-capacity precision, recall, F1 and MCC

supplement it; Appendix B.4 explains the cross-period interpretation. Across validation windows, comparisons use the paired standard error of Section 5.1. Test intervals use the weekly bootstrap of Section 5.1 (settings in Appendix Table 18), resampling purchase weeks for ranked lists (rebuilding each month's top-10% list) and review-morning weeks for the daily replay. Stability over time is summarised by each ranking's *largest shortfall*: its biggest PR-AUC gap to the best ranking in the same period, across the five validation windows and the test months.

Review budget and daily replay. The output is used as a ranking and is not cut at a probability threshold. Staff capacity is roughly fixed while the late rate swings between 4% and 21%, so a fixed threshold would flood the list in a busy period and leave it nearly empty in a quiet one. In the daily replay, staff review up to 10% of the previous day's handovers each morning at 9am, and the replay counts late parcels reached before their promised date and the days left to act. The 10% share is a comparison setting, not a staffing estimate.

5.4 Candidate models and hyperparameters

We compare linear and tree-based models, starting from plain linear or logistic regression and an unrestricted decision tree. Ridge, random forest and histogram boosting are assessed against these baselines and the operational rules using Section 5.1. The extended grids keep ridge with $\alpha = 1000$ at both regression stages and unweighted logistic regression with $C = 0.01$. Appendix B.3 explains the candidate changes and Appendix B gives the full configurations and tuning alternatives.

The proposed late-warning blend averages the model and remaining-slack rankings on fitting-sample percentile scales:

$$s_i = \tfrac{1}{2}F_{\text{model}}(p_i) + \tfrac{1}{2}F_{\text{rule}}(-r_i).$$

Here p_i is late probability and r_i is remaining slack at scoring. The 50/50 weight is fixed in advance for robustness. The evaluated monthly all-history refits refresh the classifier and percentile scales. The daily replay and scoring-time definitions are detailed in Appendix B.3 and Appendix Table 18.

6 Results and evaluation

Table 4 compares the selected models on training, validation and test data.

Table 4: Selected models across fitting, validation and test data. Training scores are in-sample results for the final fitted models; validation is the mean over five chronological windows.

Model	Metric	Train	Validation	Test
Checkout ridge	MAE (days)	5.254	5.757	5.114
	RMSE (days)	8.438	9.309	6.543
Handover ridge	MAE (days)	4.594	5.055	4.199
	RMSE (days)	7.728	8.318	5.657
Logistic, $C = 0.01$	PR-AUC (AP)	0.374	0.392	0.489
	ROC-AUC	0.813	0.780	0.863

The deployed late warning blends this model with the slack rule (Section 6.2). Training scores are in-sample; later-period regression errors fall mainly because deliveries became faster, and PR-AUC moves with prevalence, so methods are compared within each period below.

6.1 Regression results

Both stages are selected on mean validation MAE over the five chronological windows, then evaluated on the 19,230 test orders. Table 5 compares all candidates and Table 6 reports the test-period forecasts.

Table 5: Regression model comparison: mean over five chronological validation windows. MAE and RMSE are in days; lower is better. Boosting shows the better of its two settings at each stage.

	Checkout			Handover		
Model	MAE	RMSE	R^2	MAE	RMSE	R^2
Training mean	7.215	10.671	−0.078	7.215	10.671	−0.078
Route average	6.154	9.646	0.120	–	–	–
Elapsed time + route rule	–	–	–	5.304	8.396	0.337
Plain linear	5.860	9.464	0.155	5.146	8.472	0.327
Ridge, $\alpha = 1000$	**5.757**	**9.309**	**0.181**	**5.055**	**8.318**	**0.351**
Unrestricted tree	7.754	12.220	−0.455	6.888	11.951	−0.423
Tree, depth 6 / leaf 50	5.933	9.505	0.144	5.313	8.633	0.299
Random forest	5.783	9.340	0.176	5.346	8.808	0.270
Histogram boosting	5.816	9.444	0.157	5.214	8.651	0.297
Ridge–boosting average	–	–	–	5.086	8.428	0.334

6.1.1 Checkout

Model comparison and selection. Ridge with $\alpha = 1000$ has the lowest validation MAE at checkout, 5.757 days (Table 5). The random forest is second at 5.783 days, 0.026 days worse (paired SE 0.019), and gradient boosting and the depth-6 tree follow. Plain linear regression is 0.104 days worse than ridge (paired SE 0.032), more than three standard errors. Ridge is 0.397 days better than the route's average delivery time (6.154), and every candidate except the unrestricted tree improves on the training mean by more than 1.2 days.

Test performance. On the test period the checkout ridge has an MAE of 5.114 days versus 5.199 for the route average and 6.267 for the training mean. Its MAE advantage over the route rule is 0.085 days (rule minus model; 95% paired weekly-bootstrap interval $[-0.195, +0.324]$), so no clear difference is detected. It has an R^2 of −0.224 and a mean overestimate of 3.77 days (Table 6). Deliveries became much faster: mean delivery time fell from 13.3 days in training to 8.8 days in the test period, and the model, fitted on slower history, overpredicts in every test month by 2.7 to 4.5 days (Appendix Figure 4B). With the test period's own spread small (standard deviation 5.9 days), an RMSE of 6.54 days is worse than predicting that period's mean in hindsight, which a negative R^2 reflects.

Promise-free variant. On the separate 19,363-order policy test cohort, the 18-input estimate has MAE 5.37 days and mean overprediction 4.11 days. A promised date built on this estimate would largely avoid the bias, because its buffer is recalibrated each month on recent residuals (Section 8). The resulting promises are evaluated separately in Section 6.3.

What drives the estimate. The largest numeric coefficients belong to promised days, distance, freight and seller shipping history (Appendix B.4 gives magnitudes and interpretation limits).

6.1.2 Handover

Holding ridge at $\alpha = 1000$, handover reduces mean validation MAE from 5.757 to 5.055 days, a 0.702-day (12.2%) improvement, and Appendix Figure 6 shows the gain in every window. Adding route history gives 5.662 days, adding elapsed and approval days together gives 5.060, and refreshing route history at handover gives 5.055. Most of the gain comes from the order's progress so far (elapsed and approval days).

Model comparison and selection. Ridge $\alpha = 1000$ has the lowest validation MAE of the candidates (Table 5). It beats the route rule by 0.249 days (4.7%) and also has lower RMSE. Plain linear is 0.091 days worse (paired SE 0.015), and ridge is better in every window, a gap beyond the one-SE margin.

The extended grid selects an interior $\alpha = 1000$, with validation MAE 5.055 days against 5.099 at $\alpha = 100$. An equal-weight average of ridge ($\alpha = 1000$) and histogram boosting gives 5.086 days, 0.031 days worse than ridge alone (paired SE 0.019), so the ensemble is not used.

Test performance. On 19,230 later purchases, the handover ridge has MAE 4.199 days and $R^2 = 0.085$, improving on the checkout estimate's 5.114 days. In the test period ridge is 0.438 days worse than the route rule (95% paired weekly-bootstrap CI: [0.313, 0.541] days), so the validation advantage does not hold in the test period (Table 6).

Table 6: Test-period forecasts. Errors are in days; bias is prediction minus actual duration. Both ridge models use $\alpha = 1000$.

Predictor	MAE	RMSE	R^2	Bias
Checkout route average	5.199	6.479	−0.200	+3.648
Checkout ridge	5.114	6.543	−0.224	+3.768
Handover route rule	3.762	5.241	0.215	+2.447
Handover ridge	4.199	5.657	0.085	+3.120

What drives the forecast. Elapsed progress, distance and recent route transit have the largest numeric coefficients. Appendix Figure 4 gives the magnitudes, monthly bias and limits on interpreting correlated inputs.

Against the success criteria of Section 3.2, the checkout estimate is level with the route average and the handover estimate is worse than its route rule on the test period.

6.2 Classification results

The Phase 2 classifier remains the best single model, and no Phase 3 variant improves it materially. Blending it equally with the remaining-slack rule keeps most of its validation advantage and reaches late parcels with more warning time than either simple ranking. Against the success criteria of Section 3.2, the blend ranks more late parcels near the top than the rule in validation but not clearly in the test period, and reaches late parcels with more days left to act in both. The remaining-slack rule stays the formal benchmark because no ranking beats it clearly in the test period.

Model comparison and selection. Regularised logistic regression with $C = 0.01$ has the highest mean validation PR-AUC among the Phase 2 families, 0.392 (Table 7). Plain logistic regression is 0.021 worse (paired SE 0.007), three standard errors, and behind in all five windows. The best forest (0.373) and best boosting model (0.363) are also behind in every window. A

possible reason is that the slack-shortfall inputs already give the linear model the threshold a tree would otherwise have to learn. In the extended grid, PR-AUC falls steadily as regularisation strengthens, to 0.375 at $C = 0.0001$, so $C = 0.01$ is an interior choice. The best training scheme in the grid (the last 180 days) is 0.001 better, within one paired SE (0.002), so the one-standard-error rule keeps the simpler all-history fit. The deployed model is the blend of this model with the remaining-slack rule.

Table 7: Late-warning candidates: mean over five chronological validation windows. Gap is the candidate's mean PR-AUC minus that of regularised logistic regression with $C = 0.01$; paired SE is the standard deviation of the five window-by-window differences divided by $\sqrt{5}$ (Section 5.1). Each tree family shows its best setting; full grids in Appendix B.

Candidate	PR-AUC	ROC-AUC	Gap	Paired SE
Phase 2 families				
Regularised logistic, $C = 0.01$ (best validation score)	**0.392**	**0.780**	–	–
Regularised logistic, $C = 0.1$	0.386	0.769	−0.007	0.003
Plain logistic regression	0.371	0.747	−0.021	0.007
Random forest (min leaf 20, $\sqrt{p}$ features)	0.373	0.744	−0.020	0.006
Gradient boosting (rate 0.03, 200 trees, min leaf 20)	0.363	0.741	−0.029	0.007
Decision tree (depth 8, min leaf 100)	0.320	0.705	−0.072	0.009
Phase 3 variants of the selected model				
Last 180 days of training data	0.394	–	+0.001	0.002
Stronger regularisation, $C = 0.0001$	0.375	–	−0.017	0.008
Promise-relative inputs	0.391	–	−0.002	0.003
ZIP-3 and seller handover history added	0.391	–	−0.001	0.003
Fitted only on orders before their date	0.393	–	+0.001	< 0.001
Monthly refits on all history	0.393	–	+0.001	0.001
Monthly refits on the last 90 days	0.384	–	−0.008	0.006
Blend, 90% model weight	0.395	–	+0.003	0.003
Blend, 50/50 with monthly refits (deployed)	**0.383**	–	−0.009	0.006
Comparators				
Remaining-slack rule	0.328	0.697	−0.064	0.018
Regression: predicted delivery past promise	0.333	0.702	−0.059	0.019
Closest to deadline	0.225	0.581	−0.168	0.034

Phase 3 variants. No tested Phase 3 variant improves validation PR-AUC by more than 0.003, so the all-history logistic fit is retained. The 50/50 blend gives up 0.009 of validation PR-AUC (paired SE 0.006) while reducing the largest observed shortfall across periods. The regression-based ranking trails the classifier in validation (0.333 versus 0.392, paired SE of the gap 0.019). Table 7 gives the full comparison.

Validation. The blend keeps most of the model's lead over the rule: PR-AUC 0.383 against 0.328, ahead in all five windows (gap 0.055, paired SE 0.012), and 1,341 late orders caught against 1,099 (Table 8).

Test performance. On the test period, the fixed classifier trails the rule by 0.031 PR-AUC (Table 8; model minus rule; 95% interval $[-0.049, -0.005]$). At the 10% review share, the blend's list finds 61% of the 672 late test orders at 21% precision, against 10% and 3.5% for a random list of the same size. For the blend, test precision is 0.214, recall 0.612, F1 0.317 and Matthews correlation coefficient (MCC) 0.324. Appendix Table 17 reports the same metrics for

all rankings on validation and test. The blend clearly gains over the Phase 2 model, with 34 more late orders (interval $[+11, +58]$) and a PR-AUC 0.033 higher ($[+0.019, +0.047]$). Against the rule no clear difference is detected, since the intervals for late orders caught, $[-16, +26]$, and for PR-AUC, $[-0.011, +0.023]$, both include zero. Against closest to deadline the gain is clear again: 52 more late orders ($[+15, +80]$) and a PR-AUC 0.035 higher ($[+0.007, +0.093]$). The ordering does not depend on the 10% share. At review shares of 1, 2, 5 and 20%, the blend catches more late orders than the model and closest to deadline at every share, and stays within 8 of the rule (ahead at three of the four shares, behind at 5%).

Table 8: Late-warning rankings at a 10% review share: the top 10% of each validation window, or of each test purchase month (19,230 orders, 672 late). Largest shortfall is the biggest PR-AUC gap to the best of these five rankings in any validation window or test month (May 2018, with two late orders, excluded).

	Validation (W1–W5)		Test		Largest
Ranking	PR-AUC	Late caught	PR-AUC	Late caught	shortfall
Closest to deadline	0.225	596	0.487	359	0.273
Regression: predicted delivery past promise	0.333	1,122	0.479	412	0.122
Remaining-slack rule	0.328	1,099	0.519	398	0.116
Phase 2 model	0.392	1,374	0.489	377	0.037
Blend, monthly refits	**0.383**	**1,341**	**0.522**	**411**	**0.022**

Daily review queue. Re-ranking every open order each morning reaches more late parcels before their date than ranking each day's handovers once, with 256 against 241 for the blend in the test period (interval $[0, +32]$, borderline) and 1,287 against 1,118 in validation. Across rankings, the daily queue trades volume against warning time (Table 9). In the test period, closest to deadline and the rule both reach more late parcels than the blend (differences $[+74, +380]$ and $[+13, +78]$), but the blend reaches them with more days left: 2.3 more than the rule ($[+1.3, +4.1]$) and 4.7 more than closest to deadline ($[+3.5, +6.3]$). Two days is enough to warn a customer but rarely enough to change the delivery. With almost a week, staff can ask the carrier for a faster service or another recovery option. The blend's lead in parcels reached with at least three days left is not clear in the test period ($[-13, +61]$ against the rule, $[-61, +132]$ against closest to deadline) but is wider in validation. Closest to deadline therefore suits same-day customer updates and the blend suits recovery.

Table 9: Daily replay: each morning at 9am, all open orders are re-ranked and staff review up to 10% of the previous day's handovers. Counts are late orders reached before their promised date (the promised day itself counts as on time); days left are averaged over those orders. The regression ranking was not replayed.

	Validation (W1–W5)			Test		
Daily ranking	Reached	3+ days left	Mean days left	Reached	3+ days left	Mean days left
Closest to deadline	1,337	820	4.2	481	116	2.0
Remaining-slack rule	1,429	910	7.1	300	129	4.4
Blend, monthly refits	**1,287**	**1,098**	**11.8**	**256**	**152**	**6.7**

What drives the ranking. Slack shortfall, elapsed time and remaining slack have the largest validation permutation importances. Removing the three slack inputs lowers mean PR-AUC

from 0.392 to 0.378; removing relative backlog lowers it to 0.382. Appendix Figure 5 gives the full importance comparison. These are associations within the fitted model, and correlated inputs share credit.

6.3 Robustness and drift

Delivery conditions changed between the validation and test periods: mean delivery time fell from 13.3 days in training to 8.8 days in the test period.

Delivery-time estimates. Both regression stages were fitted on slower history and overestimate test delivery time in every month, the checkout estimate by 2.7–4.5 days and the handover update by 2.3–4.0 days (3.77 and 3.12 days on average; Appendix Figure 4B). The route rule has a lower test MAE than the handover ridge (3.762 against 4.199 days). The milder penalty $\alpha = 100$ also scores better at both stages (checkout 4.995, handover 3.972), but the penalty stays as selected on validation.

Late warning. Olist shortened its promises during the test period, to the shortest of any month by August 2018, and the classifier's most important inputs describe timing and the promise. No ranking is best everywhere (Appendix Table 19). The Phase 2 model ranks below the rule in every full test month; the blend has the highest PR-AUC in June and August, and the regression ranking in July. In validation the order reverses. The model is best in four of five windows, and the rule and the regression ranking fall 0.116 and 0.122 behind it in W5. Across all periods the blend's largest shortfall is 0.022, the smallest of the five rankings, against 0.037 for the model, 0.116 for the rule, 0.122 for the regression ranking and 0.273 for closest to deadline. The two parts probably fail in different conditions. The model ranks best when conditions resemble its training data, while the rule depends only on slack, which shrinks as soon as promises shorten. Monthly refits add a small gain. The refitted blend catches 411 late orders in the test period against 403 for a single fit (difference +8, 95% interval $[-2, +16]$), with a PR-AUC 0.005 higher ($[+0.002, +0.008]$), and reaches 256 against 250 in the daily replay (+6, $[+1, +11]$). Monthly refits are recommended as routine upkeep so the percentile scales follow the current order mix.

Promise buffer. The same drift means a promised date built on the checkout estimate needs a buffer that moves with recent deliveries. A buffer set once on validation, targeting 5% late, gives 33.1-day promises and 0.4% late in the test period, eleven days longer than Olist's. Recalibrating the buffer each month on the residuals of orders bought two months earlier, the latest largely complete cohort, brings the promise down to 21.6 days at 1.9% late, against Olist's 22.1 days at 3.5% (region buffer, 3.5% target). Refitting the estimate each month while keeping a fixed buffer leaves promises at 31.9 days. The drift sits in the level of the estimate, which the buffer absorbs directly, so recalibration rather than retraining does the work. The two-month lag is slow to follow sudden changes. In November and December 2017, buffers set on calmer September and October orders left about 13% of orders late, level with Olist in November (12%) but nearly twice its 7% in December, and over the whole validation period the policy only matches Olist (25.2 against 25.0 days at Olist's late rate). Peak season needs a separate guard (Section 8).

6.4 Limitations

Available data. The Olist tables lack carrier identity, tracking events and capacity data. Route history and regional backlog stand in, so the models cannot explain an individual delay or react at once to a disruption. The recorded promise, payment details and geolocation are assumed to be available at checkout, since the data hold no historical snapshots. The principal

comparisons require eventual delivery and a valid handover. The separate promise experiment requires delivery only. Cancelled and unresolved orders are outside both evaluation cohorts.

Evaluation period. The test period covers three months to August 2018, so no later cohort shows whether monthly recalibration or retraining keeps pace. Validation includes a single Black Friday peak, so the peak-season behaviour in Section 6.3 rests on one occurrence. The bootstrap measures sampling variability within the observed test period only. It cannot account for events absent from those 15 weeks or later drift, which monitoring is meant to detect. The test months were also examined during Phase 2, so later months would confirm more cleanly.

Interventions and customer response. The data record no recovery actions after a late warning, nor their costs or effects, so the results show which orders could be prioritised, not how many late deliveries an intervention would prevent. They also cannot show how customers respond to a different promised date, since a shorter promise may win orders and a longer one may lose them. The promise policy is judged on promise length and late rate alone, and choosing its target late rate needs a controlled experiment (Section 8).

Late warning. The 50/50 weight is a declared robustness choice, not the validation optimum, which favours heavier model weights. The test period contains one promise-shortening episode, so the blend's robustness to such shifts rests on a single occurrence, and its 15 weeks make the test intervals approximate. The daily replay counts late parcels reached in time, not parcels saved.

7 Model Deployment and Application

The proof of concept combines the selected models with an operations dashboard. Because the Olist data end in 2018, it replays held-out historical records instead of a live order feed. The analyst selects a date from 1 May to 18 October 2018. Overview and Ask use a midnight cutoff and the action queue uses 09:00. The promise policy starts after the Phase 2 cutoff and is shown from June 2018. Some Overview counts exclude final cancelled/unavailable statuses, so cancellation availability is approximate.

7.1 Architecture

The application exposes a JSON prediction service and a browser dashboard with Overview, Ask, Actions and Models tabs. Its registry links model runs and versions to MLflow and reports ridge with $\alpha = 1000$ for checkout and handover and logistic regression with $C = 0.01$ for late risk. The registry comparison describes the fixed Phase 2 classifier, while Actions uses the monthly-refitted blend.

The dashboard is hosted on Azure App Service at `https://olist-ops-e5abckh8h0hfahdp.eastasia-01.azurewebsites.net/`. The published promise experiment excludes both promise inputs, as specified in Section 5.2.1.

7.2 Model API

The prediction service accepts an order and replay date, returns the ETA, risk ranking and model versions, and rejects malformed requests or explains why an order cannot yet be scored. Appendix C.2 lists the endpoints and an example response.

7.3 Application walkthrough

Each tab answers one question (Figure 3).

- **Overview: how is delivery going?** Late rate, days late, reviews and deliveries over the last 30 days against the 30 before, plus overdue orders split into those with the carrier and those not yet shipped.
- **Ask: what the cards don't show.** A language model turns a plain question into one SQL query on a read-only copy of the data, limited to rows before the cutoff. The answer shows its chart, SQL and rows, and is rejected if it quotes a number that is not in the result.
- **Actions: what to do this morning.** Parcels are ranked by the blend of late risk and remaining slack (Section 5.4). The number of *Act today* slots is 10% of yesterday's handovers, rounded up, and the next same-sized block is *Watch*. Parcels already past their promise are listed for a customer update. A *Why* panel explains each rank, a *Predict an order* box calls the prediction endpoint, and a table shows the proposed promise by region.
- **Models: can we still trust them?** Validation and test results from the registry, followed by monitoring.

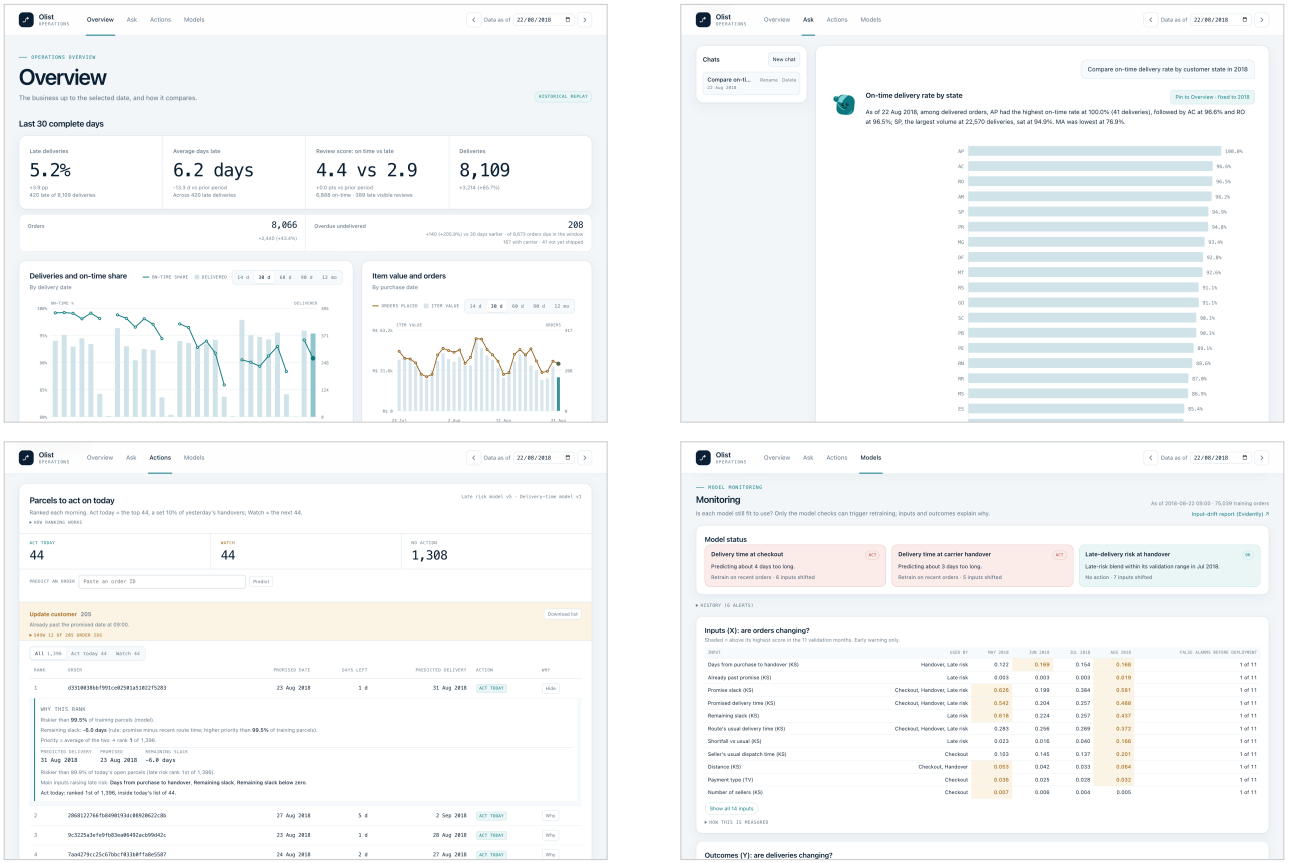


Figure 3: The dashboard on 22 August 2018. Top: Overview and Ask. Bottom: the Actions review list with one parcel's explanation open, and model monitoring.

7.4 Monitoring and maintenance

Every prediction is logged with its time, date, order, model versions, scores and response time. Monitoring asks three questions, using only outcomes known by the selected date:

- Inputs (X), are orders changing? Evidently compares each input with training (Kolmogorov–Smirnov for numbers, total variation for categories). An input is flagged when it moves further than in any of the 11 validation months, as an early warning only.
- Outcomes (Y), are deliveries changing? Delivery days, promised days and late rate, against the range seen in training months. A month counts once 95% of its orders are delivered.
- Models ($X \to Y$), are they still right? Each month's error and bias against validation (ROC-AUC for the late-risk blend), and a paired week bootstrap against the simple rule (Section 5.1).

Besides the monthly refit, only the model check triggers retraining. A model is at *Watch* when a result leaves its validation range and at *Act* when the rule is clearly better or the bias is well beyond that range. On 22 August 2018 both delivery-time models are at *Act*: they predict three to four days too long, and the handover model also loses to its route rule, while the checkout model is level with its rule. The late-risk blend raises no warning. Its ROC-AUC stays within the validation range in every judged month, and the remaining-slack rule is never clearly ahead of it. ROC-AUC is used for this check because PR-AUC moves with the late rate, which fell to about 1% in June.

The promise buffer is recalibrated and the classifier refitted monthly on all completed history (the Models tab shows the live version). A model that reaches *Act* is retrained, checked with the same validation design and registered as a new version. An analyst approves the switch before the live version changes, and the old version stays available for rollback. Until then, the simple rule is the fallback.

8 Business Recommendations and Impact

Olist should pilot three decisions: make more credible promises, give operations time to recover threatened deliveries, and contact customers proactively. Success means better service after workload and cost, not better prediction scores.

8.1 Business decisions and expected impact

Promise dates customers can rely on. Commercial and customer-experience teams should pilot the promise-free checkout estimate with a monthly regional buffer. Across 19,363 later orders, the replay gave 21.6-day promises with 1.9% lateness, versus Olist's 22.1 days and 3.5% (Section 6.3). Unrounded, promises were 0.42 days shorter and missed promises 1.56 percentage points fewer, about 156 fewer misses per 10,000 orders. Physical delivery speed is unchanged. The lagged buffer failed during the year-end peak, so Olist should keep existing promises during peaks or where regional evidence is thin until a seasonal safeguard is validated. It should promote the policy only if a concurrent pilot keeps deadlines reliable while shortening promises, at acceptable conversion and cost.

Recovery queues matched to action time. Logistics operations should allocate intervention slots by urgency and feasible recovery time. At equal daily capacity, the blend reached 256 late

parcels with 6.7 days' mean warning and closest-to-deadline ranking reached 481 with 2.0 days (Table 9). For actions that need several days, logistics should pilot the blend against deadline and remaining-slack rules, ranking all eligible open parcels each morning. The blend's count with at least three days remaining was higher, 152 versus 116, but statistically uncertain. The pilot should start at the evaluated 10% of yesterday's handovers, then set capacity from staffing and choose the queue that prevents more delays per intervention and per staff-hour.

Messages and escalation. Orders at the top of the review queue should alert logistics and customer service, who should verify shipment status, assess carrier escalation, and tell affected customers the revised arrival estimate and next action. The handover route rule outperformed ridge on the later cohort (Table 6), so customer updates should use the estimator validated for current conditions. Staff should keep the original promise alongside revised estimates. If delivery is late, the platform or seller may offer a voucher under an agreed eligibility and cost policy, and seller accountability requires operational evidence.

8.2 Monitoring and maintenance under drift

Analytics should review error, bias, ranking performance and outcome coverage weekly. Monthly all-history classifier refits are maintenance candidates. Regression replacements should be promoted only after they outperform the current model and route rule on subsequent orders, and promise-buffer recalibration should be assessed separately from model refitting.

Recently completed orders overrepresent fast deliveries. Analytics should track unresolved shipments separately, never assume they are on time, and wait for enough follow-up before judging an update.

8.3 Pilot roadmap and promotion criteria

Olist should begin in shadow operation to check feature availability, queue feasibility and staff workload. It should then run concurrent controlled pilots, first separating promise changes from recovery and customer messaging. Allocation, follow-up and acceptable trade-offs should be set beforehand, with repeat customers and shared carrier capacity accounted for. The combined workflow comes afterwards, because changing promises also changes risk inputs and the lateness target.

The pilots should measure promise length, missed-promise rate and conversion; delays prevented at equal intervention capacity; and complaints, ratings, repeat purchases and voucher expenditure. Throughout, Olist should record model and buffer versions, original promises, actions, communications, costs and outcomes, then calculate incremental benefit after staff time, carrier charges and compensation, since flagged orders show exposure, not savings. It should scale only after adequate follow-up shows acceptable service and cost across regions and peak periods.

Declaration on the Use of AI

AI assistance was used to write and review code, audit numerical outputs, and draft prose. The team remains responsible for checking the source data, methods, computed results, citations and final submission.

References

[1] C. Bergmeir and J. M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi:10.1016/j.ins.2011.12.028.

[2] G. James, D. Witten, T. Hastie and R. Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 1st edition, 2013. doi:10.1007/978-1-4614-7138-7.

[3] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi:10.1371/journal.pone.0118432.

[4] N. Salari, S. Liu and Z.-J. M. Shen. Real-time delivery time forecasting and promising in online retailing: when will your package arrive? *Manufacturing & Service Operations Management*, 24(3):1421–1436, 2022. doi:10.1287/msom.2022.1081.

[5] V. Deshpande and P. K. Pendem. Logistics performance, ratings, and its impact on customer purchasing behavior and sales in e-commerce platforms. *Manufacturing & Service Operations Management*, 25(3):827–845, 2023. doi:10.1287/msom.2021.1045.

[6] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi:10.1023/A:1010933404324.

[7] L. Grinsztajn, E. Oyallon and G. Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems*, 35:507–520, 2022. doi:10.52202/068431-0037.

[8] A. C. de Araujo and A. Etemad. End-to-end prediction of parcel delivery time with deep learning for smart-city applications. *IEEE Internet of Things Journal*, 8(23):17043–17056, 2021. doi:10.1109/JIOT.2021.3077007.

[9] B. Ü. Küp, E. T. Küp, G. Koçak, A. Yücekaya and M. Hekimoğlu. Real-time prediction of delivery delay in supply chains using machine learning approaches. SSRN preprint, 2024. doi:10.2139/ssrn.5054862.

[10] T. O. Hodson. Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. *Geoscientific Model Development*, 15:5481–5487, 2022. doi:10.5194/gmd-15-5481-2022.

[11] J.-G. Gaudreault and P. Branco. Empirical analysis of performance assessment for imbalanced classification. *Machine Learning*, 113:5533–5575, 2024. doi:10.1007/s10994-023-06497-5.

Appendix A Feature List

Table 10: The 20 checkout inputs underlying the principal model comparisons. The separate promise-policy variant removes promised days and promise slack.

Group	Input	Definition
Promise	`promised_days`	Days from purchase to Olist's estimated delivery date.
	`promise_slack`	Promised days minus the route's typical delivery days.
Route	`distance_km`	Haversine distance between seller and customer ZIP-prefix centroids.
	`same_state`	1 if seller and customer are in the same state.
	`route_typical_days`	Smoothed mean delivery days on the state-to-state route.
	`seller_state`	Seller's state (furthest seller for multi-seller orders).
	`customer_state`	Customer's state.
Seller history	`seller_ship_days`	Smoothed mean purchase-to-handover days of the seller's earlier orders.
Order contents	`n_items`	Number of items.
	`n_products`	Number of distinct products.
	`n_sellers`	Number of distinct sellers.
	`total_price`	Item price total (R$).
	`total_freight`	Freight charged at checkout (R$).
	`total_weight_g`	Total product weight (g).
	`total_volume_cm3`	Total product volume (cm^3).
	`product_category`	Category of the order's largest item by volume.
Payment	`payment_type`	Method of the order's first payment.
	`max_installments`	Largest number of instalments.
Timing	`purchase_hour`	Hour of purchase.
	`purchase_dayofweek`	Day of week of purchase.

Table 11: Inputs added at carrier handover.

Input	Used by	Definition
`handover_days`	Both	Days from purchase to carrier handover.
`approval_days`	Both	Days from purchase to payment approval; unknown if approved after handover.
`route_transit_90d_at_handover`	Both	Smoothed mean carrier time on the route over deliveries completed in the 90 days before handover (prior strength 20).
`remaining_slack`	Classifier	Promised days minus handover days minus recent route transit.
`past_promise_at_handover`	Classifier	1 if handover days exceed promised days, comparing timestamps at promise midnight; this is distinct from being past the promised calendar date.
`slack_shortfall`	Classifier	Days by which remaining slack is below zero; 0 otherwise.
`state_backlog_recent_vs_8wk`	Classifier	Log of one plus current open orders minus log of one plus their mean over eight earlier weekly snapshots, for the buyer's state; the scored order is excluded.

Appendix B Full Model Configurations

B.1 Regression (checkout and handover)

Checkout and handover use the same model candidate settings but different operational baselines: the route's historical average delivery time at checkout, and elapsed days plus recent route carrier time at handover. All models are fitted with scikit-learn 1.9.1, pandas 3.0 and NumPy 2.5; the exact versions are recorded with each saved output. Ridge is tuned over the extended grid below.

Table 12: Checkout and handover regression configurations.

Model	Settings and evaluated alternatives
Training mean	`DummyRegressor(strategy='mean')`.
Checkout route average	Smoothed historical mean purchase-to-delivery time on the seller-state–customer-state route, using only deliveries recorded before purchase. Uses all available prior history, with smoothing towards the overall historical mean (prior strength 20). Missing values use the fitting-data median of the route-history feature.
Handover route rule	Elapsed purchase-to-handover days plus the route's 90-day mean carrier transit time, using only delivery outcomes recorded before handover. Missing route values use the fitting-data median. Route smoothing prior strength: 20.
Plain linear	`LinearRegression`; fitted intercept; no penalty.
Ridge	Fitted intercept; squared-error objective with L_2 penalty; `solver='auto'`, `tol=0.0001`, `max_iter=None`, `positive=False`, seed 42. Search $\alpha \in \{0.01, 0.1, 1, 10, 100, 1000, 10000, 100000\}$; $\alpha = 1000$ selected at both stages.
Decision tree	`(max_depth, min_samples_leaf)` alternatives: `(None,1)` and `(6,50)`. Squared-error criterion; best splitter; `min_samples_split=2`.
Random forest	200 trees; `max_depth=None`; `min_samples_leaf=20`; `max_features='sqrt'`; bootstrap sampling; squared-error criterion; `n_jobs=4`.
Histogram boosting	Baseline `(learning_rate,max_iter,min_samples_leaf)` = `(0.1,100,20)`; slower alternative `(0.05,300,40)`. Squared-error loss; 31 maximum leaf nodes; no depth cap; 255 bins; L_2 penalty 0.
Equal-weight ensemble	Arithmetic mean of ridge ($\alpha = 1000$) and baseline histogram boosting; reference comparison, not a selectable candidate.

Additional effective tree settings. Decision trees and the forest use seed 42, no pruning (`ccp_alpha=0`), no minimum impurity decrease, zero minimum weighted leaf fraction, no maximum leaf-node cap and no monotonic constraints. Decision trees consider all features at a split. The forest uses `min_samples_split=2`, `max_samples=None`, `oob_score=False`, `warm_start=False` and `verbose=0`.

Additional effective boosting settings. Both variants use seed 42 and `max_features=1.0`, with automatic early stopping (10% validation fraction, ten iterations without improvement, tolerance 10^{-7}). Numeric inputs are median-imputed and categorical inputs ordinal-encoded (unknown code -1).

Linear pre-processing. The `log1p` transforms apply to total price, freight, seller–buyer distance, product weight and volume. Imputation and scaling are fitted within each training window; the purchase-hour sine/cosine transformation and one-hot category encoding are also inside the pipeline. These settings are unchanged across the ridge search.

Candidate results and ridge search. Table 13 lists every handover candidate and Table 14 the full ridge search at both stages. Both use the same features and chronological windows as Table 5. Ridge with $\alpha = 1000$ has the lowest validation MAE at checkout and at handover and is selected at both stages.

Table 13: Handover candidates: mean metrics across five validation windows. Duration errors are in days; lower is better.

Candidate	MAE	RMSE	R^2
Training mean	7.215	10.671	−0.078
Elapsed time + route transit rule	5.304	8.396	0.337
Plain linear regression	5.146	8.472	0.327
Ridge, $\alpha = 1$	5.145	8.470	0.327
Ridge, $\alpha = 10$	5.136	8.460	0.329
Ridge, $\alpha = 100$	5.099	8.406	0.338
Ridge, $\alpha = 1000$ (selected)	**5.055**	8.318	0.351
Unrestricted decision tree	6.888	11.951	−0.423
Decision tree, depth 6	5.313	8.633	0.299
Random forest, minimum leaf 20	5.346	8.808	0.270
Histogram boosting, baseline	5.214	8.651	0.297
Histogram boosting, slower learning	5.215	8.666	0.294

Table 14: Extended ridge search at checkout and handover, using unchanged features and windows. Mean over five chronological validation windows; errors in days.

	Checkout		Handover	
Candidate	MAE	RMSE	MAE	RMSE
Plain linear	5.860	9.464	5.146	8.472
Ridge, $\alpha = 0.01$	5.860	9.464	5.146	8.472
Ridge, $\alpha = 0.1$	5.860	9.463	5.146	8.472
Ridge, $\alpha = 1$	5.858	9.462	5.145	8.470
Ridge, $\alpha = 10$	5.848	9.451	5.136	8.460
Ridge, $\alpha = 100$	5.809	9.398	5.099	8.406
Ridge, $\alpha = 1000$	**5.757**	**9.309**	**5.055**	**8.318**
Ridge, $\alpha = 10000$	5.801	9.338	5.142	8.418
Ridge, $\alpha = 100000$	6.205	9.733	5.822	9.211

Test-period uncertainty. The interval for the handover test MAE gap in Section 6.1.2 (Table 6) is ridge minus route rule. We resample the 15 purchase calendar weeks (Monday–Sunday, retaining partial boundary weeks) 2,000 times with replacement, using seed 42. Each draw uses the same weeks for both methods and retains all orders in each sampled week; Each method's MAE is calculated over those orders and ridge minus rule is recorded. The 2.5th and 97.5th percentiles of the 2,000 paired differences give the 95% interval. Predictions remain fixed, so the interval describes variation in the observed mix of weeks, not uncertainty from model refitting or future changes in delivery conditions.

B.2 Classification (late-order warning)

All classifiers share the 27 handover inputs (Appendix Tables 10 and 11), the five validation windows of Table 3 and seed 42. Pipelines are fitted on each window's fitting rows only. Linear models median-impute numeric inputs, apply $\log(1 + x)$ to price, freight, distance, weight and

volume, standardise, encode purchase hour as a sine–cosine pair and one-hot encode categories (unknown categories ignored). Trees and forests median-impute and one-hot encode without scaling. Boosting median-imputes numeric inputs and passes categories as ordinal codes to the model's native categorical handling.

Table 15: Classifier configurations and tuning grids (Phase 2). Each family's first entry is its plain baseline.

Model	Settings and evaluated alternatives
Plain logistic	`LogisticRegression`, no penalty ($C = \infty$); `solver='lbfgs'`, `max_iter=3000`.
Regularised logistic	L_2 penalty, $C \in \{0.01, 0.1, 1, 10\}$, each with `class_weight` `None` or `'balanced'`; same solver. Selected: $C = 0.01$, unweighted.
Decision tree	Plain baseline (`max_depth=None`, `min_samples_leaf=1`); grid `max_depth` $\in \{4, 6, 8, 12\}$ × `min_samples_leaf` $\in \{20, 100\}$. Gini criterion, best splitter.
Random forest	200 trees, `max_depth=None`, bootstrap sampling, `n_jobs=4`; grid `min_samples_leaf` $\in \{5, 20, 50\}$ × `max_features` $\in \{$`'sqrt'`$, 0.5\}$.
Gradient boosting	`HistGradientBoostingClassifier`, early stopping off, native categorical features; grid `learning_rate` $\in \{0.03, 0.1\}$ × `max_iter` $\in \{200, 500\}$ × `min_samples_leaf` $\in \{20, 100\}$.

Table 16: Every Phase 2 classifier candidate: mean PR-AUC and ROC-AUC over the five validation windows. "bal" marks balanced class weights.

Candidate	PR-AUC	ROC-AUC	Candidate	PR-AUC	ROC-AUC
Logistic, plain	0.371	0.747	Tree, depth 12, leaf 20	0.299	0.654
Logistic, $C = 10$	0.373	0.749	Tree, depth 12, leaf 100	0.314	0.669
Logistic, $C = 10$, bal	0.367	0.744	Forest, leaf 5, $\sqrt{p}$	0.369	0.737
Logistic, $C = 1$	0.377	0.754	Forest, leaf 5, half	0.354	0.723
Logistic, $C = 1$, bal	0.369	0.747	Forest, leaf 20, $\sqrt{p}$	0.373	0.744
Logistic, $C = 0.1$	0.386	0.769	Forest, leaf 20, half	0.364	0.730
Logistic, $C = 0.1$, bal	0.375	0.755	Forest, leaf 50, $\sqrt{p}$	0.372	0.747
Logistic, $C = 0.01$	**0.392**	**0.780**	Forest, leaf 50, half	0.369	0.737
Logistic, $C = 0.01$, bal	0.385	0.772	Boost, rate 0.03, 200, leaf 20	0.363	0.741
Tree, plain	0.140	0.567	Boost, rate 0.03, 200, leaf 100	0.353	0.734
Tree, depth 4, leaf 20	0.272	0.654	Boost, rate 0.03, 500, leaf 20	0.358	0.736
Tree, depth 4, leaf 100	0.269	0.654	Boost, rate 0.03, 500, leaf 100	0.343	0.725
Tree, depth 6, leaf 20	0.296	0.690	Boost, rate 0.1, 200, leaf 20	0.345	0.732
Tree, depth 6, leaf 100	0.305	0.701	Boost, rate 0.1, 200, leaf 100	0.338	0.725
Tree, depth 8, leaf 20	0.301	0.691	Boost, rate 0.1, 500, leaf 20	0.336	0.717
Tree, depth 8, leaf 100	0.320	0.705	Boost, rate 0.1, 500, leaf 100	0.329	0.715

B.2.1 Family comparisons and sensitivity results

Phase 3 variants. No variant improves mean validation PR-AUC by more than 0.003 over the Phase 2 model (Table 7, middle panel), a small fraction of the model's 0.064 lead over the rule. Stronger regularisation, refits on only the last 90 days and adding both finer delivery-history inputs all lower it. None therefore changes how the model compares with the rule, and the

simpler Phase 2 model is kept. Ranking by the regression's predicted lateness is no substitute for a classifier: it trails the model in every window (0.333 against 0.392; gap 0.059, paired SE 0.019) and is only slightly ahead of the rule (0.328; 1,122 late orders caught against 1,099), so it was ruled out in validation (Table 8). In the test period it is not clearly different from the blend: it catches 412 late orders against 411 (interval for the difference $[-16, +16]$), and the blend's PR-AUC is 0.043 higher, not a clear difference ($[-0.003, +0.082]$). Across all periods, though, its largest shortfall is 0.122, in W5, against 0.022 for the blend. A likely reason for its validation gap is that the regression is fitted to get delivery times right on average, so two parcels expected to arrive two days early score the same even if one travels a route that often runs a week over; the classifier is fitted to the late outcome itself and can rank the riskier parcel higher. The blend is the exception in kind rather than size: it gives up 0.009 of validation PR-AUC (paired SE 0.006) in exchange for robustness to promise changes, which validation cannot test.

Table 17: Quality of the 10% review list: precision, recall, F1 and MCC, pooled over the five validation windows (38,003 orders, 4,053 late) and over the test months (19,230 orders, 672 late). Orders on the list count as predicted late. A random list scores precision equal to the late rate, recall 10% and MCC zero.

	Validation (W1–W5)				Test			
Ranking	Precision	Recall	F1	MCC	Precision	Recall	F1	MCC
Closest to deadline	0.157	0.147	0.152	0.054	0.187	0.534	0.277	0.275
Regression	0.295	0.277	0.286	0.204	0.214	0.613	0.317	0.325
Remaining-slack rule	0.289	0.271	0.280	0.197	0.207	0.592	0.307	0.312
Phase 2 model	0.361	0.339	0.350	0.275	0.196	0.561	0.290	0.292
Blend, monthly refits	**0.353**	**0.331**	**0.341**	**0.266**	**0.214**	**0.612**	**0.317**	**0.324**

Table 18: Phase 3 variants and evaluation settings. All variants use the selected logistic model ($C = 0.01$) unless stated; results are in Table 7.

Item	Settings
Regularisation and recency	$C \in \{0.0001, 0.0003, 0.001, 0.003, 0.01\}$, on all history each also with balanced class weights, × training on all history, the last 365 or 180 days, or all history with recency weights $0.5^{\text{age}/h}$ (half-life h of 90 or 180 days). One-standard-error rule, simplest scheme first.
Class weights	Mean validation PR-AUC on all history, unweighted / balanced: $C = 0.0001$: 0.3749 / 0.3818; $C = 0.0003$: 0.3812 / 0.3857; $C = 0.001$: 0.3877 / 0.3883; $C = 0.003$: 0.3913 / 0.3884; $C = 0.01$: 0.3923 / 0.3854.
Promise-relative inputs	Six inputs (promised days, promise slack, elapsed days, recent route transit, remaining slack, slack shortfall) replaced by three ratios: elapsed days / promised days, remaining slack / promised days, recent route transit / typical route days; ratios clipped.
More delivery history	Added: 90-day carrier time to the customer's three-digit ZIP area (prior strength 20) and the seller's 90-day handover speed (prior strength 10), both using only events before the order's handover.
Recoverable orders	Fitted only on orders before their promised date at handover; orders already past their date ranked above all others.
Blend	$s = w\,F_{\text{model}}(p) + (1 - w)\,F_{\text{rule}}(-r)$, percentiles against the fitting orders. Fixed weights $w \in \{0.1, 0.2, \ldots, 0.9\}$; recommended $w = 0.5$.
Promise-varying weight	Rule weight = min(1, 2× share of orders bought in the previous 30 days with a promise below the fitting orders' 10th percentile); the cut-off is 14.52 days for the full training set.
Monthly refits	Refit on the first day of each month on orders delivered by then: all history (recommended), or the last 90 or 180 days; percentile scales refreshed.
Review list	Top 10% of each validation window or test purchase month; also 1, 2, 5 and 20%.
Daily replay	9am each morning; capacity = 10% of the previous day's handovers, rounded up; open orders before their promised date (promised day counts as on time) re-ranked; an order is reviewed at most once.
Intervals	Test: paired bootstrap over 15 calendar weeks, 2,000 resamples, seed 42, 2.5th–97.5th percentiles. Validation: paired standard error over five windows.
Regression ranking	Handover ridge ($\alpha = 1000$, Appendix B.1) predicted delivery days minus promised days, largest first.

Table 19: Late-warning PR-AUC period by period. The best ranking in each period is in bold. Late rates differ, so compare rankings within a row, not down a column. May 2018 (755 orders, two late) is omitted.

Period	Late rate	Closest to deadline	Regression	Rule	Model	Blend
W1	4.1%	0.277	0.361	0.354	0.378	**0.388**
W2	13.8%	0.253	0.362	0.343	**0.430**	0.411
W3	4.8%	0.194	0.261	0.256	**0.282**	0.281
W4	9.9%	0.170	0.303	0.301	**0.370**	0.356
W5	20.7%	0.229	0.380	0.386	**0.502**	0.480
June 2018	1.2%	0.121	0.194	0.188	0.165	**0.200**
July 2018	3.4%	0.388	**0.466**	0.448	0.429	0.449
August 2018	6.2%	0.606	0.534	0.620	0.597	**0.632**
Largest shortfall		0.273	0.122	0.116	0.037	**0.022**

B.3 Candidate design and sensitivity rationale

B.3.1 Regression: checkout

Model families. Linear models (plain linear regression and ridge) are compared with tree-based models (decision tree, random forest [6] and histogram gradient boosting), against the training mean and the route's average delivery time. Linear models suit the checkout estimate: its strongest inputs, distance and the route's typical delivery time, act roughly additively, the fitted coefficients can be checked against operational sense, and refitting is cheap and stable. Ridge adds an L_2 penalty, which shrinks correlated inputs together rather than choosing among them [2]. That matters here: freight, weight, volume and distance move together, promised days overlap with the route's typical delivery time, and the two state variables and product category expand into more than a hundred one-hot columns, many with few orders. Trees can capture thresholds and interactions, such as a distance effect that differs by region, without engineered inputs.

Tuning grids. Ridge is tuned over α. The decision tree is fitted unrestricted and at depth 6 with a minimum leaf of 50 orders; the forest uses 200 trees with a minimum leaf of 20 and $\sqrt{p}$ features per split; boosting is run at learning rate 0.1 with 100 iterations and a minimum leaf of 20, and at a slower 0.05 with 300 iterations and a minimum leaf of 40. Following the course baseline principle, each family starts from its simplest variant, plain linear regression and the unrestricted tree, and the one-standard-error rule in Section 5.1 ranks all configurations in one list from simplest to most complex. Full settings are in Appendix B.1.

Starting point and diagnosis. Phase 2 tuned ridge over $\alpha \in \{1, 10, 100\}$. The forest had the lowest validation MAE (5.783 days), but ridge with $\alpha = 100$ was within one paired standard error (gap 0.026, SE 0.028) and simpler, so ridge was selected. Two questions were left open. Validation MAE was still falling at $\alpha = 100$, the edge of the grid, so the best penalty had not been found. And on the test period the checkout estimate overpredicted by 3.65 days on average, as deliveries became faster than in training. Phase 3 makes two changes in response.

- **Extended penalty grid.** α is searched from 0.01 to 100,000 in factors of ten (Appendix Table 14). Validation MAE is lowest at $\alpha = 1000$ (5.757 days) and rises on both sides (5.809 at $\alpha = 100$, 5.801 at $\alpha = 10{,}000$), so the optimum now lies inside the grid. The value is large in absolute terms, but each window fits 23,664 to 52,166 orders on standardised inputs, and the penalty falls mostly on the sparse one-hot columns.
- **Regularisation tested against the plain baseline.** Phase 2 applied the one-standard-error rule only between ridge and the forest. Here it is also applied between ridge and plain linear regression, the simplest variant of the linear family: plain linear regression would be kept if its disadvantage were within one paired standard error of ridge.

Promise-free fit. For promise setting, the same ridge is refitted without promised days and promise slack (Section 5.2.1). It uses the selected $\alpha = 1000$ rather than a separate search, so that the estimate behind any proposed promise is the one evaluated here, minus Olist's promise.

B.3.2 Regression: carrier handover

We compare linear models (plain linear and ridge) and tree-based models (decision tree, forest and histogram boosting), against training-mean and elapsed-time-plus-route baselines. A ridge–boosting average tests ensemble value.

As at checkout, the Phase 2 ridge grid was extended after Phase 2 feedback, to $\alpha \in \{0.01, 0.1, 1, 10, 100, 1000,$ and $\alpha = 1000$ is selected on validation MAE. The information ladder holds this penalty fixed at

every stage to isolate the value of additional information. Ridge fits squared error plus an L_2 penalty and MAE selects the configuration. Plain linear regression would be preferred only if it fell within one paired standard error of ridge, following the rule in Section 5.1. Appendix B.1 gives full settings and comparisons.

B.3.3 Classification: late-order warning

Model families. Linear models (plain and regularised logistic regression) are compared with tree-based models (decision tree, random forest and gradient boosting). Linear models suit a ranking task with few late orders per window: they are stable, quick to refit and their coefficients can be checked against operational sense. Trees can capture thresholds and interactions without engineered inputs. Logistic regression is tuned over the inverse regularisation strength C, with and without class weights, and each tree family over depth or leaf size and, for ensembles, features per split, number of trees or learning rate (full grids in Appendix Table 15).

Starting point and diagnosis. Phase 2 selected regularised logistic regression with $C = 0.01$. It ranked better than the remaining-slack rule in all five validation windows but worse in the test period (PR-AUC 0.489 against 0.519), mostly in August 2018, when 56% of orders had promises under 14.5 days compared with about 10% in training. A monthly refit, checked in Phase 2, narrowed the gap only slightly. Phase 3 tests five ways to close that gap, each aimed at a possible cause.

- **Regularisation and recency.** The Phase 2 grid, $C \in \{0.01, 0.1, 1, 10\}$, had its best value on the boundary. It is extended downwards with $C \in \{0.0001, 0.0003, 0.001, 0.003\}$, and these values and $C = 0.01$ are crossed with five training schemes: all history, the last 365 or 180 days, and recency weighting with half-lives of 90 or 180 days. Stronger shrinkage or recent data could make the model less tied to conditions in its training period.
- **Promise-relative inputs.** Slack and elapsed time are expressed as shares of the promise, so that a shorter promise changes the inputs in proportion rather than moving them outside the range seen in training.
- **More delivery history.** Two further inputs, carrier time over the last 90 days to the customer's three-digit ZIP area and the seller's recent handover speed, add finer-grained history than the state-to-state route.
- **Training on recoverable orders.** The model is fitted only on orders still before their promised date at handover, the orders staff can still act on; orders already past their date are placed at the top of the list.
- **Model–rule blend.** The model and the rule fail in different conditions. The rule depends only on slack, which shrinks as soon as Olist shortens its promises, so it adjusts to that kind of shift at once. The blend averages the two rankings on a common scale:

$$s_i = \tfrac{1}{2}\, F_{\text{model}}(p_i) + \tfrac{1}{2}\, F_{\text{rule}}(-r_i),$$

  where p_i is the model's late probability for order i, r_i its remaining slack (missing route transit filled with the fitting median), and each F is the share of fitting orders with a score at or below the given value. Percentiles put a probability and a number of days on one scale; averaging the raw scores is not possible, because the rule produces no probability. Measuring percentiles against the fitting orders, rather than against the other parcels in the same day's list, means any single parcel can be scored the moment it is handed over, so the same score can drive a live dashboard.

Why equal weights. Validation PR-AUC rises with the model's weight, from 0.383 at 50% to 0.395 at 90%. The validation windows, however, contain no promise-shortening episode, so a weight chosen on them would favour the conditions in which the model already wins. The reported comparisons use a fixed 50/50 weight as a hedge between rankings that fail in different conditions; this is a declared robustness choice. A weight that varies with promise length was tested and not adopted (Appendix B).

Monthly refits and daily re-ranking. The evaluated maintenance strategy refits the classifier monthly on all outcomes available then and refreshes its percentile references. It is a candidate operating policy, not proof of an optimal retraining frequency. In the daily replay, eligible parcels were handed over before that morning's midnight, remain undelivered at 09:00, and have not passed their promised calendar date. Previously reviewed parcels are excluded. The model percentile assigned at handover is retained; the rule percentile uses the most recent reference distribution available at the review date. Remaining slack at review is $\min(r_i, d_{it})$, where r_i is handover slack and d_{it} is the time left before the promise. Slack need not fall every day: it remains unchanged until time left falls below handover slack. Capacity is the previous day's handover count multiplied by 10% and rounded up.

B.4 Model interpretation and metric diagnostics

What drives the estimate. Among the standardised numeric inputs, the largest mean absolute coefficients are promised days (1.52), distance (1.24), freight (0.87) and the seller's typical shipping time (0.61). The estimate therefore leans on how far the parcel travels and how quickly the seller usually ships, as well as on Olist's own promise. As at handover, these coefficients describe the fitted model, not causal effects.

What drives the forecast. Elapsed time has the largest mean absolute numeric coefficient (3.307), followed by log distance (1.301), route transit (0.891) and promise slack (0.771): the model responds to progress while retaining route and promise information (Figure 4A). These coefficients describe the fit, not causal effects; correlated inputs and omitted categorical coefficients limit importance comparisons.

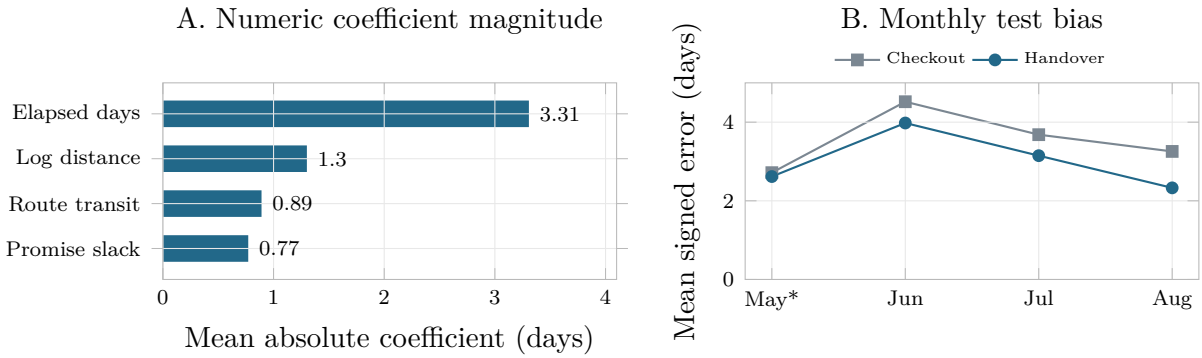


Figure 4: Handover ridge ($\alpha = 1000$) interpretation and drift. A: top four numeric inputs, averaged over validation fits after scaling; categorical coefficients are excluded. B: checkout and handover forecasts (both $\alpha = 1000$) overestimate duration in each test purchase month. May* covers purchases from 26 May only.

What drives the ranking. Permutation importance on validation measures how much PR-AUC falls when one input is shuffled (Figure 5). The largest drops are for slack shortfall (0.083), elapsed time (0.056), remaining slack (0.032), recent route transit (0.030) and promised

days (0.026). These inputs describe elapsed progress, route duration and the promise; every other input has a smaller drop. Two ablations confirm the engineered inputs: without the three slack inputs, mean validation PR-AUC falls from 0.392 to 0.378, and without relative backlog to 0.382, in all five windows each time. This reliance on the promise is consistent with the model's weaker result once Olist shortened its promises (Section 6.3). Importance describes the fitted model, not causal effects, and correlated inputs share credit.

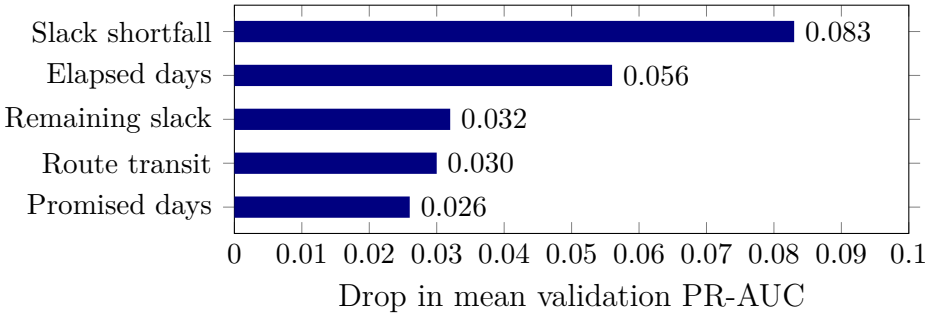


Figure 5: Late warning: the five inputs whose shuffling most lowers mean validation PR-AUC (permutation importance, Phase 2 classifier). These inputs describe progress, route duration and the promise.

Why PR-AUC rather than ROC-AUC. Both metrics summarise a ranking across all list sizes, but they answer different questions. ROC-AUC measures how often a late order is ranked above an on-time one. With 79–97% of orders on time, most of those comparisons are against easy on-time orders far down the list, so ROC-AUC can look strong even when the short list staff actually review is mostly on time [3]. PR-AUC is built from precision and recall, so it rewards placing late orders at the top, which is the decision operations faces. Its drawback is that it depends on the late rate: a random ranking scores a PR-AUC equal to the period's late rate. The Phase 2 model's PR-AUC of 0.50 in W5 is 2.4 times that window's late rate, while its 0.49 in the test set is 14 times; similar values hide very different skill. Top-10% recall has the same problem and is also capped when more than 10% of orders are late: in W5, a perfect list of 760 orders could find at most 48% of the 1,576 late orders. ROC-AUC does not depend on the late rate and is reported alongside, but selection uses PR-AUC.

B.4.1 Information available at handover

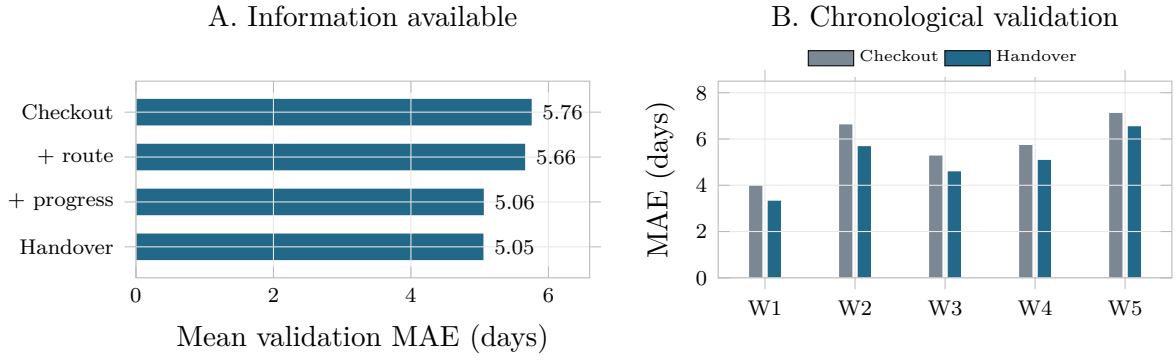


Figure 6: Handover information improves MAE in every window (ridge $\alpha = 1000$, same orders; penalty held fixed across stages). A: add route history, add elapsed/approval days jointly, then refresh route history at handover. B: five chronological validation windows.

Appendix C Supporting Data and Interface Details

C.1 Source tables

Table 20: Olist tables and how they are used.

Table	Rows	Use in this project
`orders`	99,441	Timestamps, status, promised date; targets
`order_items`	112,650	Item counts, price, freight, seller per item
`order_payments`	103,886	Payment type and instalments
`order_reviews`	99,224	Review scores, for the business case only
`customers`	99,441	Customer state, ZIP prefix, `customer_unique_id`
`sellers`	3,095	Seller state and ZIP prefix
`products`	32,951	Category, weight and dimensions
`geolocation`	1,000,163	ZIP-prefix coordinates for distance
`category_name_translation`	71	English category names

C.2 Prediction service interface

Every model endpoint is a read-only `GET` that takes the replay date as `date=YYYY-MM-DD` (Table 21). Bad dates and malformed order IDs are rejected with a JSON error before any model runs, and every prediction names the model versions behind it.

Table 21: Prediction service endpoints.

Endpoint	Returns
`/api/actions/predict`	One order's late-risk percentile, predicted delivery, remaining slack, rank that morning and main risk inputs.
`/api/actions/queue`	The 09:00 review list.
`/api/actions/promises`	Proposed promise and buffer by region.
`/api/models`, `/api/models/monitor`	Registry results and monitoring status.

```
GET /api/actions/predict?date=2018-08-22&order_id=d3310038bbf991ce02501a51022f5283
{"ok": true, "rank": 1, "tier": "act", "late_risk_percentile": 0.995,
 "predicted_delivery_date": "2018-08-31", "promised_date": "2018-08-23",
 "model_versions": {"checkout_eta": 1, "handover_eta": 1, "late_risk": 5}, ...}
```

An order that cannot be scored yet, for example one not handed over or already delivered, gets a short status message instead.